OAKLAND COMMUNITY COLLEGE®

ROBOTICS/AUTOMATED SYSTEMS TECHNOLOGY

ROB 2140
Studio 5000 and FANUC Robotics

John Sefcovic
Oakland Community College

TABLE OF CONTENTS

export to create a library

LAB: CREATE NEW ROBOT ETHERNET/IP CONNECTIONS
PROGRAM FILE: FANUC_Robot
OBJECTIVE: Create the Ethernet module and the FANUC communications node.

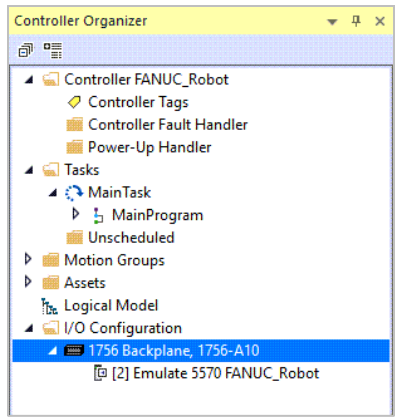
Open the project file.

Filename: FANUC_Robot

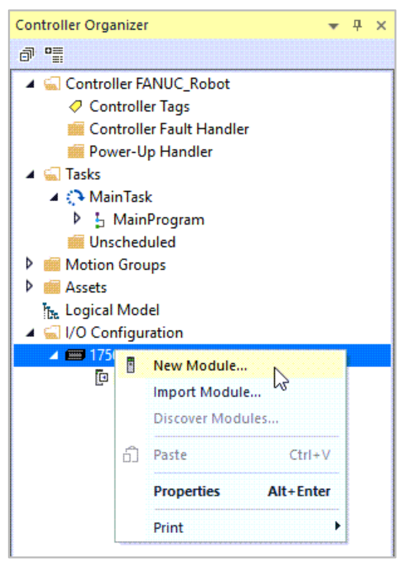
Adding Ethernet/IP Communications Module

Note: different models of processor may have built in Ethernet/IP connections and would not need to be added. The communication properties would still need to be configured.

Right click on **1756 Backplane, 1756-A10**.

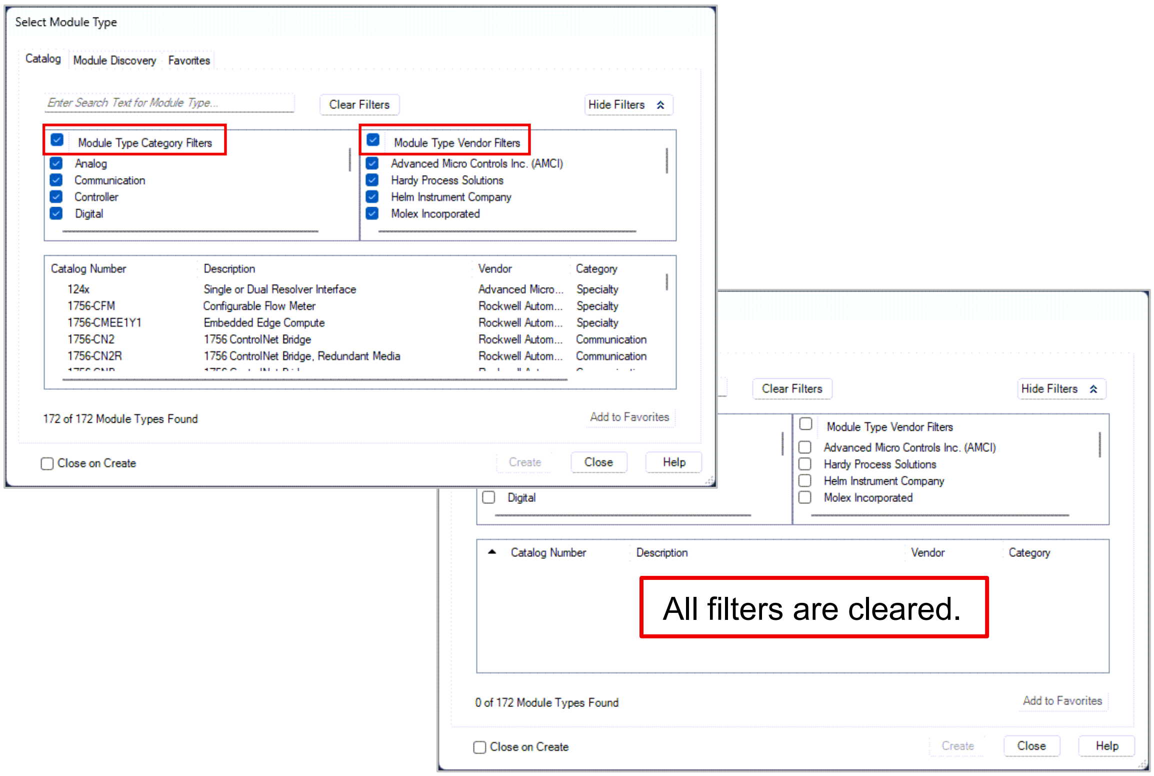


Select **New Module…**



The **Select Module Type** dialog opens.

Uncheck **Module Type Category Filters**.
Uncheck **Module Type Vendor Filters**.



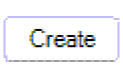
Select the following:

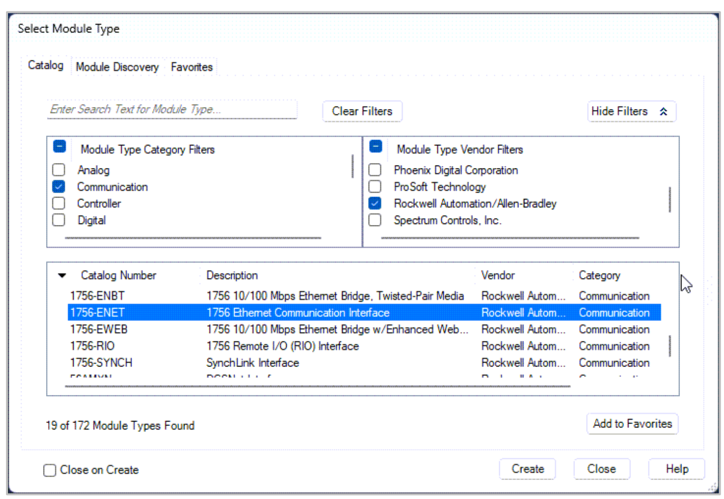
In **Module Type Category Filters** select **Communications**.

In **Module Type Vendor Filters** scroll down and select **Rockwell Automation/Allen-Bradly**.

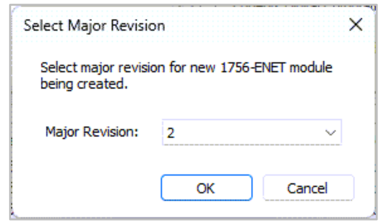
In the listing select **1756-ENET** the **1756 Ethernet Communication Interface**.

Select the **Create** button.





Select the **OK** button for the **Select Major Revision**.

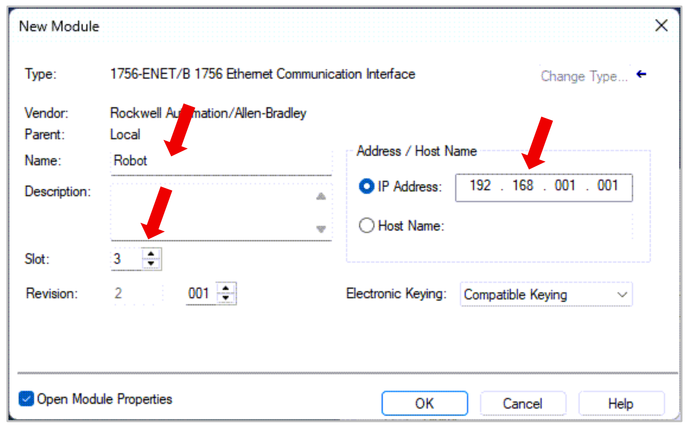


The **New Module** dialog opens.

Fill in the following:

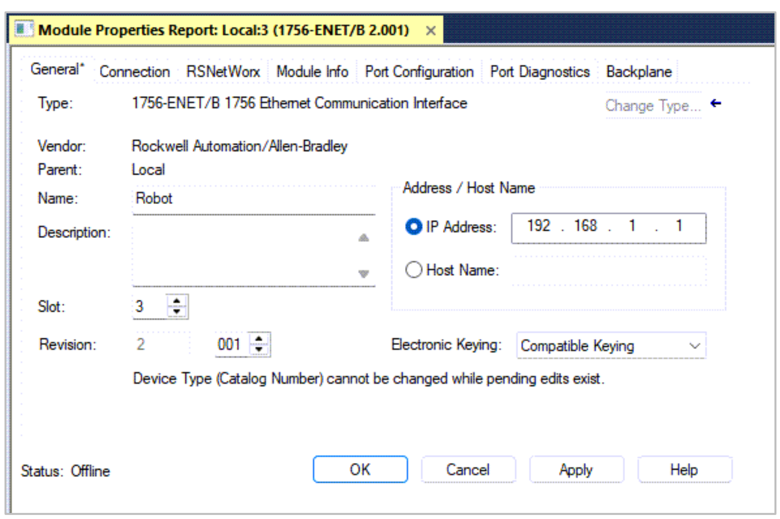
Name: Robot
Slot: 3
IP Address: 192 168 001 001 1
Do not type in spaces

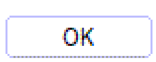
Select the **OK** button. 



The module's **Properties** dialog opens.

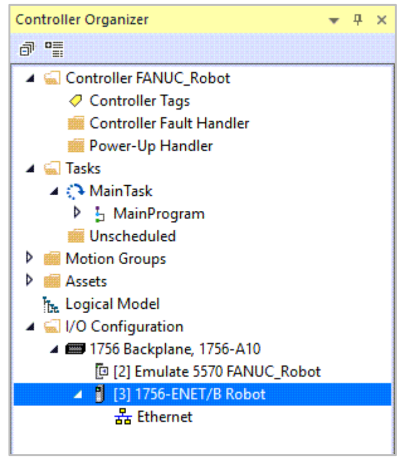
Select the **General** tab and verify that the correct configuration has been entered.



Select the **OK** button. 

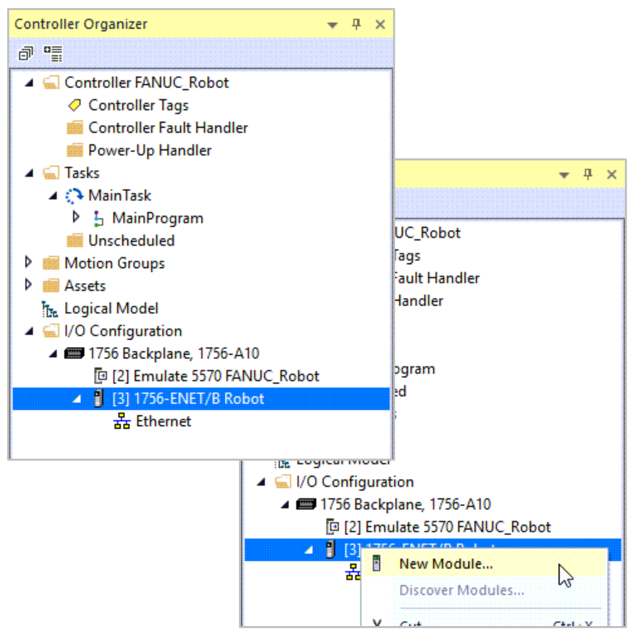
The **1756-ENET** module for Ethernet/IP communications named **Robot** has been added to the **I/O Configuration**.

Some hardware has built in ethernet settings



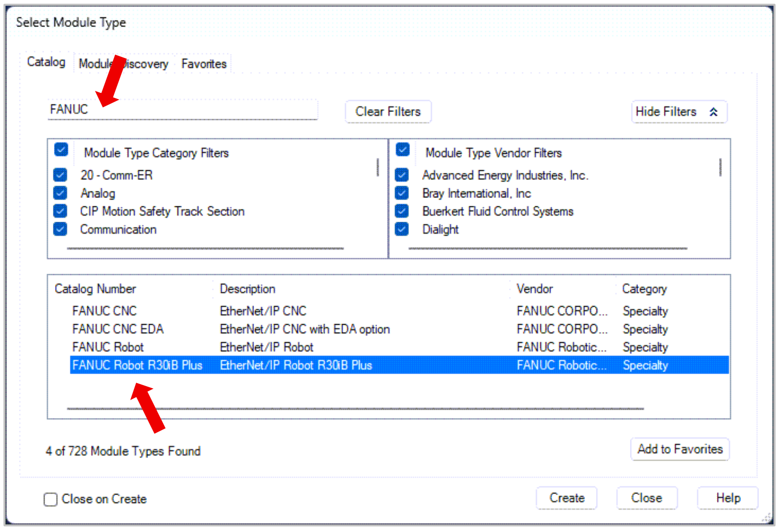
Adding FANUC Communications Node

Right Click on **[3] 1756-ENET/B Robot** and select **New Module**.

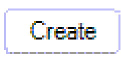


continue

In the Filter field *Enter Search Text for Module Type…* enter **FANUC**.



Select **FANUC Robot R30iB Plus** for Ethernet/IP communications.

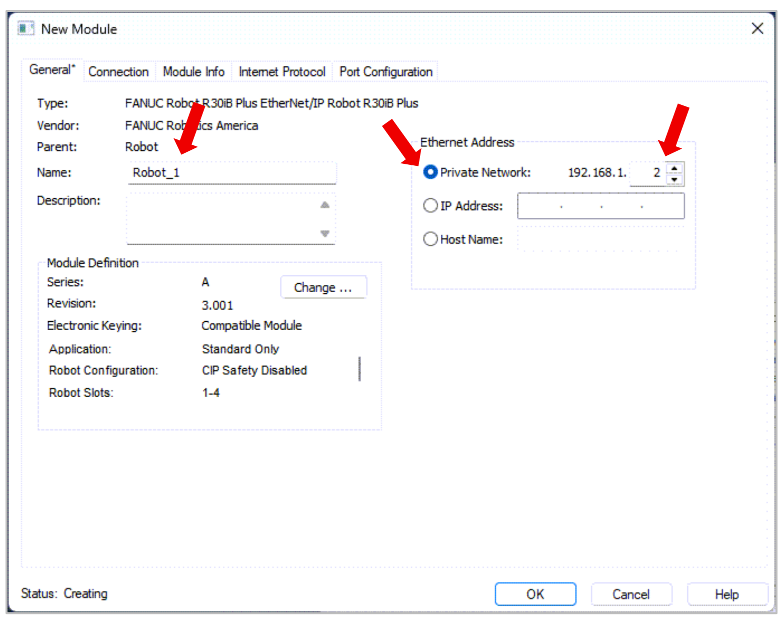
Select the **Create** button. 

The **New Module** dialog opens.

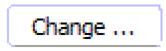
Change the following:

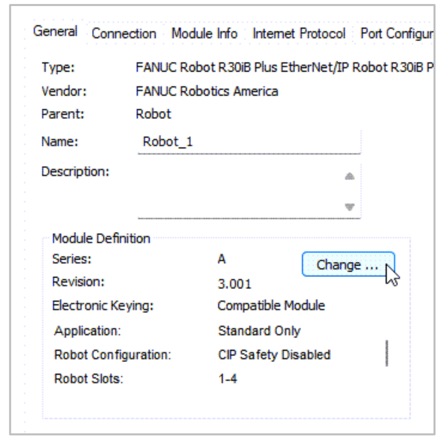
Name: Robot_1
Select **Private Network:**
Change IP Address to **2**

Set up communication protocols



In the **Module Definition** select the **Change…** button.





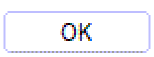
Select the **Connection** tab.

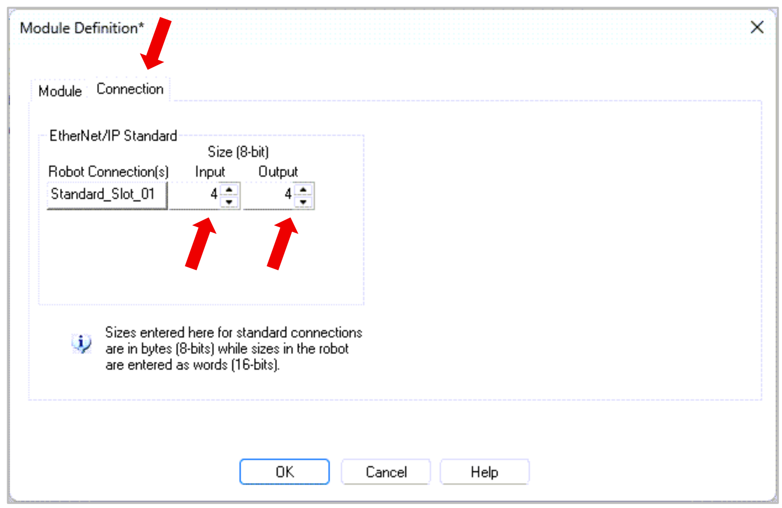
Change **Size (8-Bit)** for:

Input to **4**

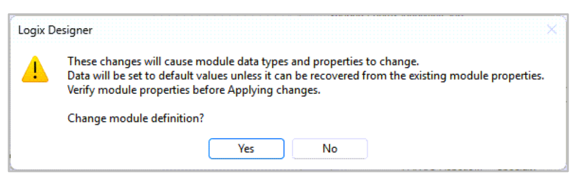
Output to **4**

Select the **OK** button.



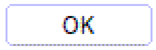


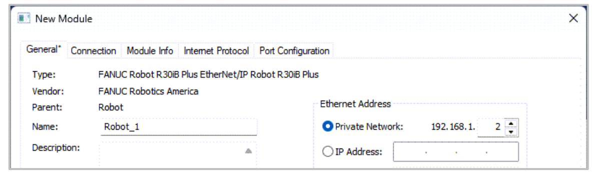
At the prompt for Change Module definition? select the **Yes** button.



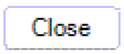
Continue

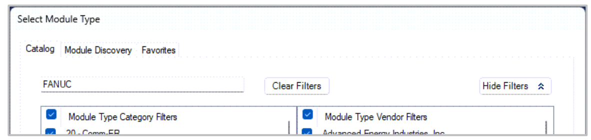
At the New Module select the **OK** button.



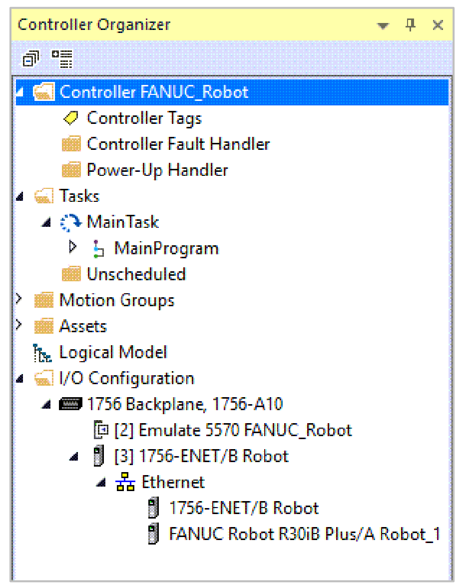


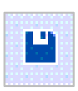
At the Select Module Type select the **Close** button.





The **FANUC Robot R30iB Plus/A Robot_1** is added to the **Ethernet** connection in the **I/O Configuration**.

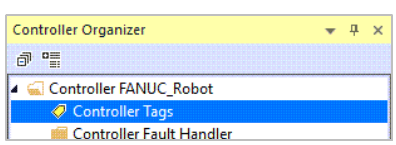




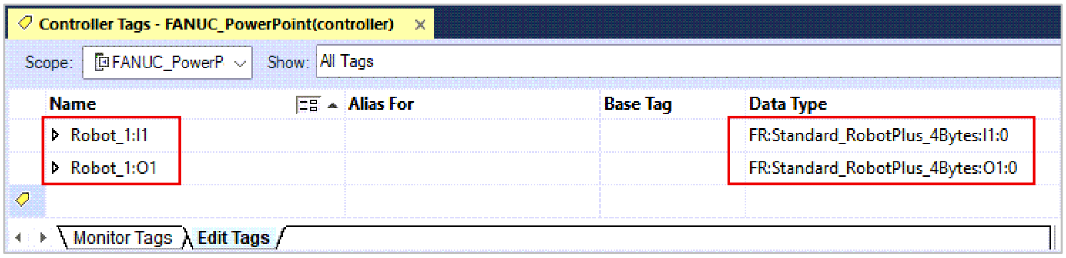
Save the PLC program file.

INFORMATION - View the Controller Tag Structure for the FANUC I/O

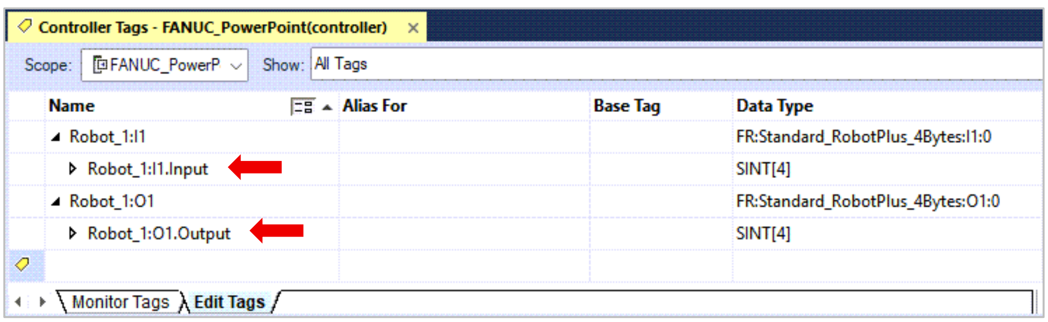
Open the **Controller Tags**.



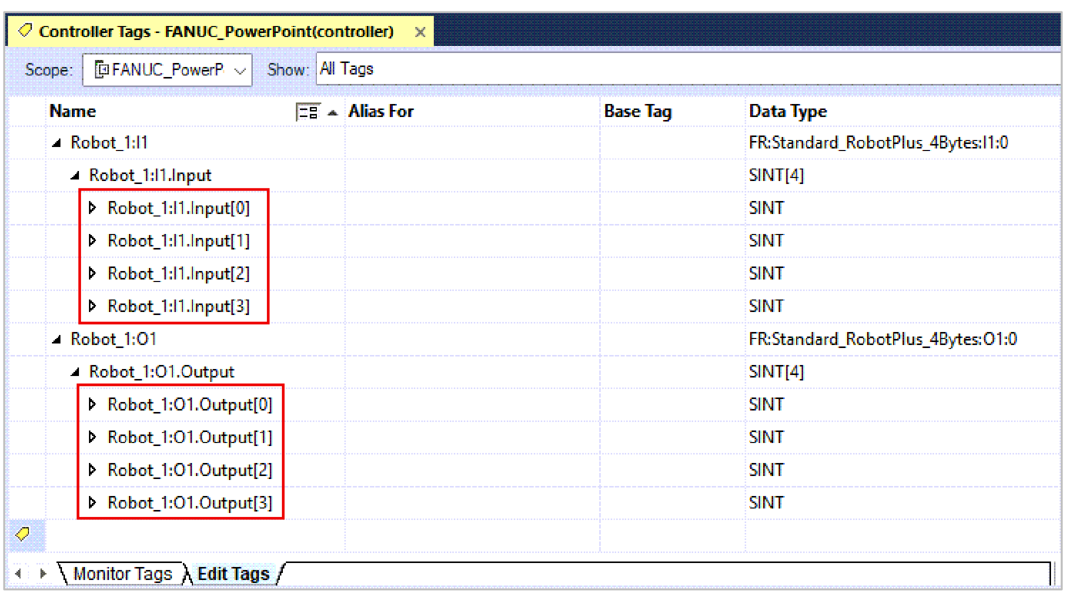
The **Robot_1** is listed as **I** (Input) and **O** (Output) as a **Data Type** of **4 Bytes** in an array of **FR:Standard_RobotPlus**.



Expand **Robot_1:1I** and **Robot_1:O1** which are a 4 element array of the **SINT[4]** (Single Integer) data.

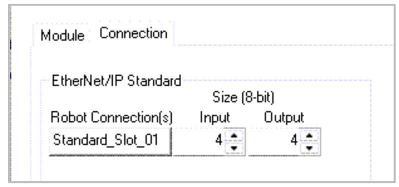


Expand **Robot_1:1I.Input** and **Robot_1:O1.Output** and the four elements of **SINT** (Single Integer) data words, **[0]** to **[4]**, are shown.



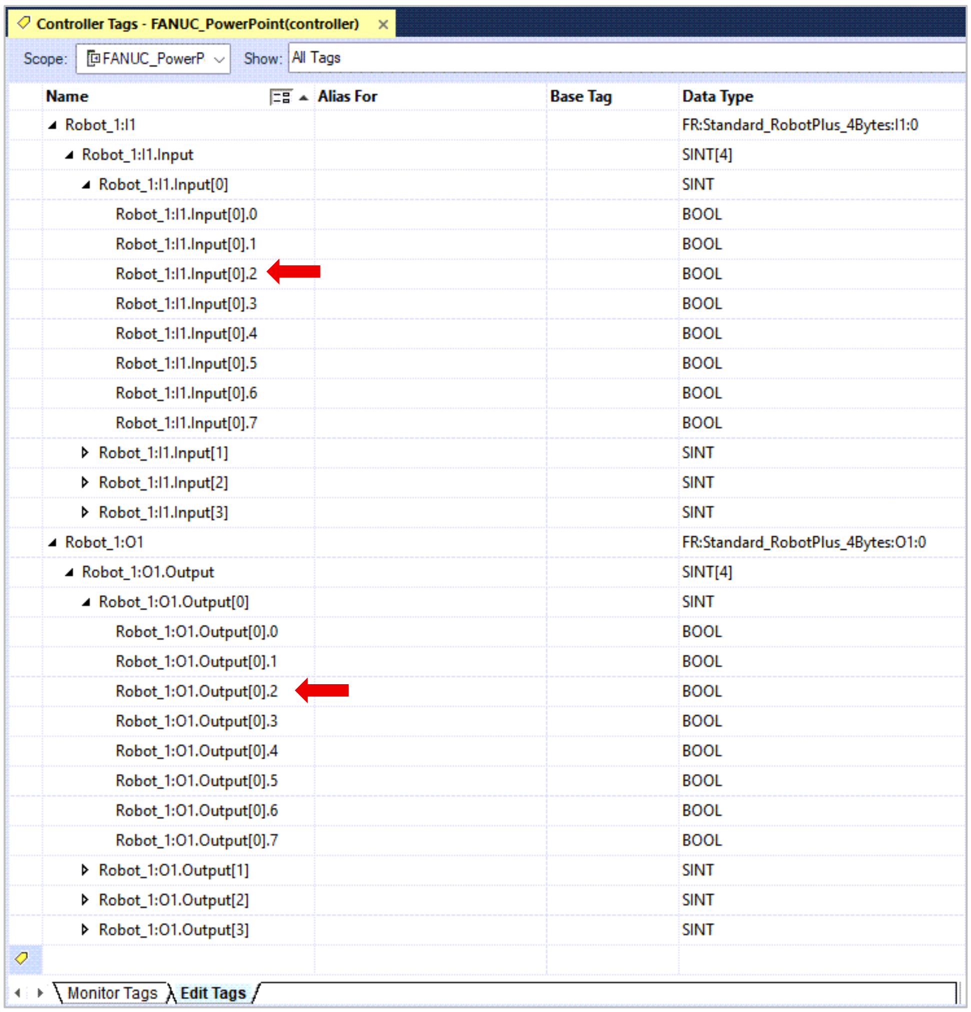
continue

The configured **Size (Byte)** of **4** when defining the **Module Definition** configuration.

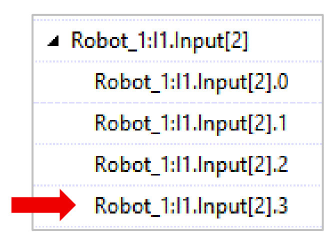


Expand **Robot_1:1I.Input[0]** and **Robot_1:O1.Output[0]**, the individual **BOOL** (Bit) address **.0** to **.7** are shown.

Examples: **Robot_1:I1.Input[0].2**
Robot_1:O1.Output[0].2



When expanding **Robot_1:I1.Input[2]** the third word in the array, element **2**, the address for the bit would be **Robot_1:I1.Input[2].3**



FOR GRADE

Open Ethernet/IP module **Robot_1 Properties** and show in the General tab the defined settings.

Open the **Module Definition** and show the Input and Output size.

output plc go to robot inputs
robot outputs go to robot inputs

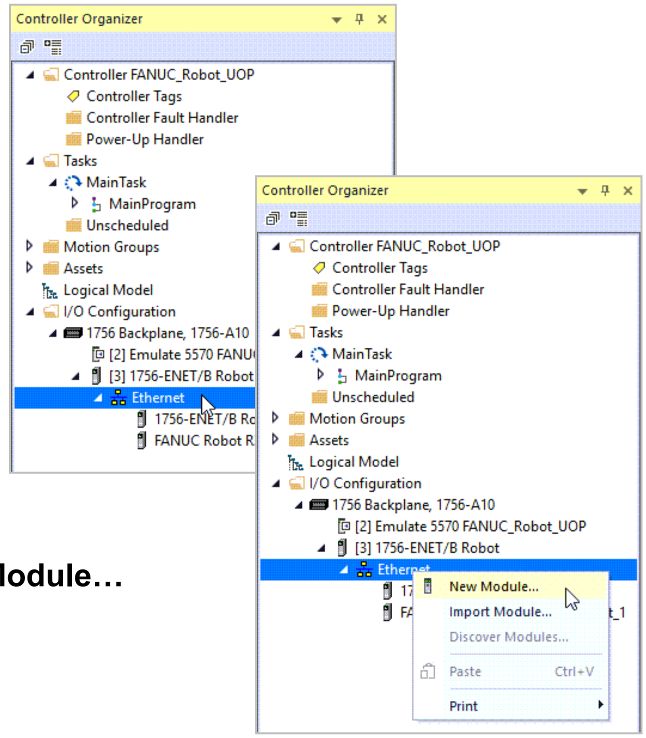
PLC Input is a robot output

LAB: CREATE NEW MACHINE ETHERNET/IP CONNECTIONS.
PROGRAM FILE: FANUC_Robot_UOP.ACD
OBJECTIVE: Add Ethernet/IP node for Machine for Robot connections.

Add profile to communicate with machine

Right click on **Ethernet**.

Select **New Module…**



Uncheck **Module Type Category Filters** and **Module Type Vendor Filters**.

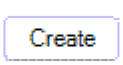
Select the following:

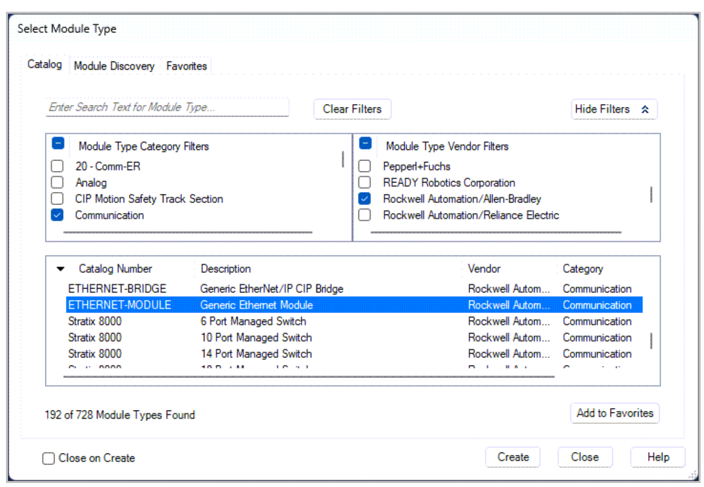
In **Module Type Category Filters** select **Communications**.

In **Module Type Vendor Filters** scroll down and select **Rockwell Automation/Allen-Bradly**.

In the listing select **ETHERNET-MODULE**.

Select the **Create** button.



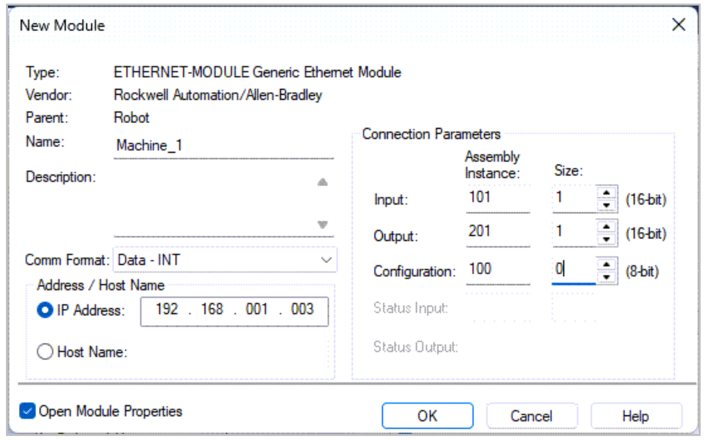


On plc side 7 items to type in. On robot side only a couple
This is a generic Module

The **New Module** dialog opens.

Change the following:

Name: Machine_1
Comm Format: Data - INT
IP Address: 192 168 001 003
Do not type in spaces
Connection Parameters
Input: 101 **Size:** 1
Output: 201 **Size:** 1
Configuration: 100 **Size:** 0



Select the **OK** button. OK

you need manufactures documentation for settings

Robot has 64 network channels

The module's **Properties** dialog opens.

Select the **General** tab and verify that the correct configuration has been entered.

Select the **OK** button. OK

INFORMATION - View the Controller Tag Structure for the Machine

Similar to the Robot, the **Machine _1** is listed as **I** (Input) and **O** (Output) as a **Data Type** of **4 Bytes** in an array of **AB:ETHERNET_MODULE**.

▸ Machine_1:I			AB:ETHERNET_MODULE_INT_2Bytes:I:0
▸ Machine_1:O			AB:ETHERNET_MODULE_INT_2Bytes:O:0
▸ Machine_1:C			AB:ETHERNET_MODULE:C:0

A **C***onfiguration array for communications is created.*

Expand **Machine_1:1I** and **Machine_1:O1** are a 1 element array of the **INT[1]** (Integer) data.

◢ Machine_1:I			AB:ETHERNET_MODULE_INT_2Bytes:I:0
▸ Machine_1:I.Data			INT[1]
◢ Machine_1:O			AB:ETHERNET_MODULE_INT_2Bytes:O:0
▸ Machine_1:O.Data			INT[1]

continue

Expand **Machine_1:1I.Input** and **Machine_1:O1.Output** and one element of **INT** (Integer) data, **[0]**, are shown.

◢ Machine_1:I			AB:ETHERNET_MODULE_INT_2Bytes:I:0
◢ Machine_1:I.Data			INT[1]
▷ Machine_1:I.Data[0]			INT
◢ Machine_1:O			AB:ETHERNET_MODULE_INT_2Bytes:O:0
◢ Machine_1:O.Data			INT[1]
▷ Machine_1:O.Data[0]			INT

The I/O "On Machine" hardware has 16 Inputs, **.0** to **.15** bit addresses.

The expanding would show 16 Outputs, **.0** *to* **.15** *bit addresses.*

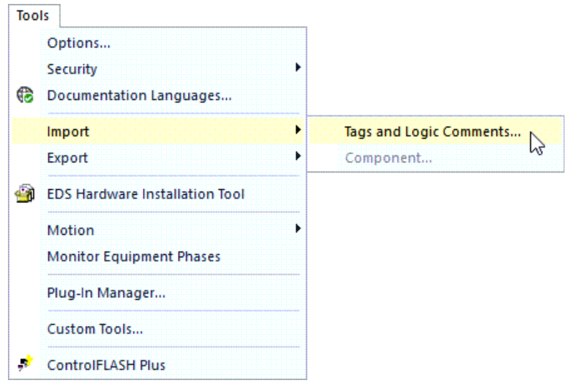
◢ Machine_1:I			AB:ETHERNET_MODULE_INT_2Bytes:I:0
◢ Machine_1:I.Data			INT[1]
◢ Machine_1:I.Data[0]			INT
Machine_1:I.Data[0].0			BOOL
Machine_1:I.Data[0].1			BOOL
Machine_1:I.Data[0].2			BOOL
Machine_1:I.Data[0].3			BOOL
Machine_1:I.Data[0].4			BOOL
Machine_1:I.Data[0].5			BOOL
Machine_1:I.Data[0].6			BOOL
Machine_1:I.Data[0].7			BOOL
Machine_1:I.Data[0].8			BOOL
Machine_1:I.Data[0].9			BOOL
Machine_1:I.Data[0].10			BOOL
Machine_1:I.Data[0].11			BOOL
Machine_1:I.Data[0].12			BOOL
Machine_1:I.Data[0].13			BOOL
Machine_1:I.Data[0].14			BOOL
Machine_1:I.Data[0].15			BOOL
◢ Machine_1:O			AB:ETHERNET_MODULE_INT_2Bytes:O:0
◢ Machine_1:O.Data			INT[1]
▷ Machine_1:O.Data[0]			INT

FOR GRADE

Show the Tag Editor **Machine_1**, Data Word 2, Bits 0 to 15.

LAB: IMPORT TAGS
PROGRAM FILE: FANUC_Robot
OBJECTIVE: Import the Tag for a .CSV spreadsheet file.

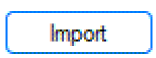
On the Menu Bar select **Tools - Import - Tags and Logix Comments**.

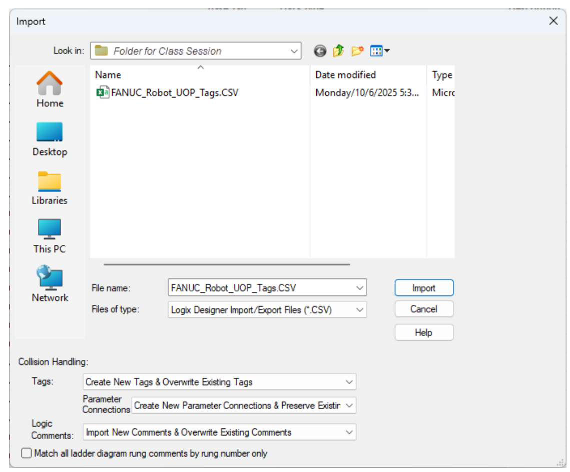


Navigate to the folder for the *Class Session* and **Robot**

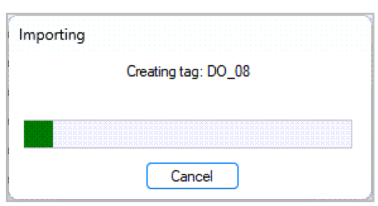
Select the file FANUC_Robot_UOP_Tags.CSV

Select the **Import** button.





The Import progress is shown.

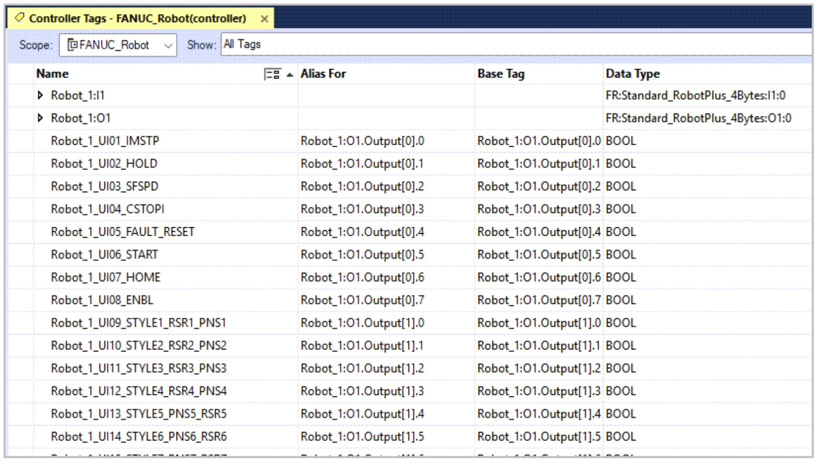


Continue

Stop is an alias or hardware address.

If scope column is blank controller tag

The Tags for the **UOP UI** and **UO** (User Operator Panel), **DI** and **DO** (Digital), and **GI** and **GO** (Group) Inputs and Outputs are imported into the application

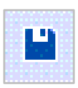


Controller Tags - FANUC_Robot(controller)

Scope: FANUC_Robot Show: All Tags

Name	Alias For	Base Tag	Data Type
Robot_1:I1			FR:Standard_RobotPlus_4Bytes:I1:0
Robot_1:O1			FR:Standard_RobotPlus_4Bytes:O1:0
Robot_1_UI01_IMSTP	Robot_1:O1.Output[0].0	Robot_1:O1.Output[0].0	BOOL
Robot_1_UI02_HOLD	Robot_1:O1.Output[0].1	Robot_1:O1.Output[0].1	BOOL
Robot_1_UI03_SFSPD	Robot_1:O1.Output[0].2	Robot_1:O1.Output[0].2	BOOL
Robot_1_UI04_CSTOPI	Robot_1:O1.Output[0].3	Robot_1:O1.Output[0].3	BOOL
Robot_1_UI05_FAULT_RESET	Robot_1:O1.Output[0].4	Robot_1:O1.Output[0].4	BOOL
Robot_1_UI06_START	Robot_1:O1.Output[0].5	Robot_1:O1.Output[0].5	BOOL
Robot_1_UI07_HOME	Robot_1:O1.Output[0].6	Robot_1:O1.Output[0].6	BOOL
Robot_1_UI08_ENBL	Robot_1:O1.Output[0].7	Robot_1:O1.Output[0].7	BOOL
Robot_1_UI09_STYLE1_RSR1_PNS1	Robot_1:O1.Output[1].0	Robot_1:O1.Output[1].0	BOOL
Robot_1_UI10_STYLE2_RSR2_PNS2	Robot_1:O1.Output[1].1	Robot_1:O1.Output[1].1	BOOL
Robot_1_UI11_STYLE3_RSR3_PNS3	Robot_1:O1.Output[1].2	Robot_1:O1.Output[1].2	BOOL
Robot_1_UI12_STYLE4_RSR4_PNS4	Robot_1:O1.Output[1].3	Robot_1:O1.Output[1].3	BOOL
Robot_1_UI13_STYLE5_PNS5_RSR5	Robot_1:O1.Output[1].4	Robot_1:O1.Output[1].4	BOOL
Robot_1_UI14_STYLE6_PNS6_RSR6	Robot_1:O1.Output[1].5	Robot_1:O1.Output[1].5	BOOL

Not all Tags will be used in the assignments.



Save the PLC program file.

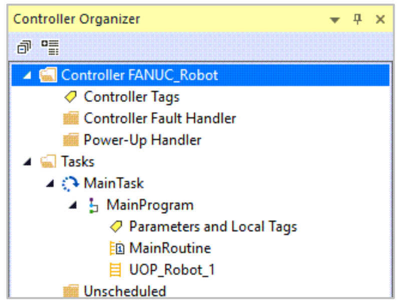
FOR GRADE

Show the Imported tags.

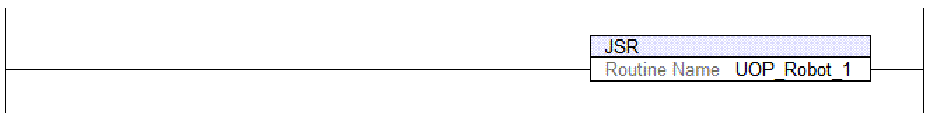
Set up required output, if you download this all outputs would be on.

LAB: ADD REQUIRED USER OPERATOR PANEL SIGNALS
PROGRAM FILE: FANUC_Robot
OBJECTIVE: Add the required UOP outputs for robot operations.

Create a subroutine **UOP_Robot_1**.



In the **Main Routine** call the subroutine **UOP_Robot_1** without any parameters.



In the **UOP_Robot_1** routine add the following Tags to and Output Energize (OTE) instruction.

There are **no input instructions**; the Outputs will be the only instruction on the unconditional rung and use only one output per rung. The Inputs will be added in other lab assignments.

OUTPUT: The Tags were previously imported and **do not** need to be created.

Alias Address	Controller Tag	Description
Robot_1:O1.Output[0].0	Robot_1_UI01_IMSTP	Immediate Stop (1)
Robot_1:O1.Output[0].1	Robot_1_UI02_HOLD	Hold Robot (1)
Robot_1:O1.Output[0].2	Robot_1_UI03_SFSPD	Safety Speed (1)
Robot_1:O1.Output[0].3	Robot_1_UI04_CSTOPI	Controlled Stop (1)
Robot_1:O1.Output[0].4	Robot_1_UI05_FAULT_RESET	Fault Reset (1)
Robot_1:O1.Output[0].7	Robot_1_UI08_ENBL	Enable Motion (1)

All these signals require an ON or Logic 1 for the robot to operate.

(1) See APPENDIX B: FANUC USER OPERATOR PANEL ASSIGNMENTS for compete description.

Save the PLC program file.

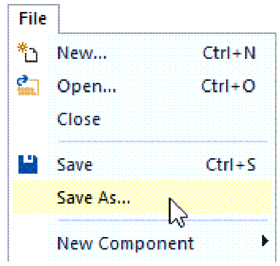
FOR GRADE

Show the **UOP_Robot_1** routine.

export the file

LAB: SAVE AS AN EXPORT FILE
PROGRAM FILE: FANUC_Robot
OBJECTIVE: Export the PLC program as a Library File.

On the Menu Bar select **File - Save As…**

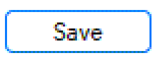


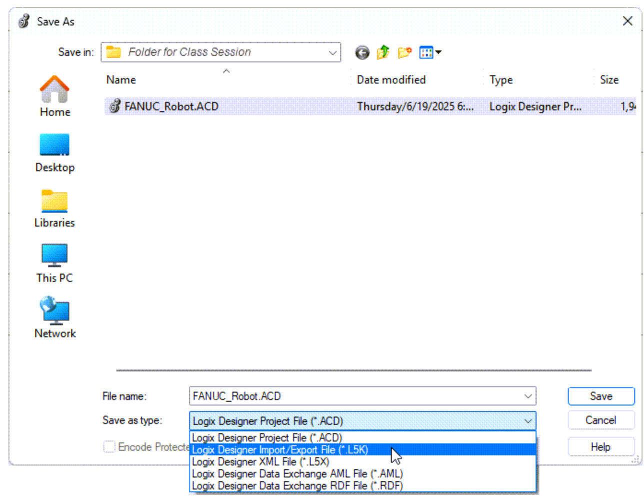
export as xml file .L5k
Have the file open FYI

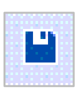
Select the **FANUC_Robot.ACD**

From the **Save as type:** dropdown select **Logix Designer Import/Export File (*.L5K).**

Select the **Save** button.

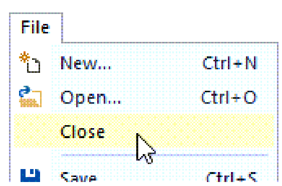


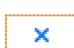




Save the PLC program file.

On the Menu Bar select **File** - **Close** Studio



Use either the **Exit**  icon or Menu bar **File** - **Exit**.

The **.L5K** file is a text code version of the program's configuration, tags, and logic that may be used to store the project in a **Library** as a **Template** for other PLC program projects.

When a .L5K file is opened the filename must be changed to preserve the original file.

Individual Programs and Subroutine may be exported in the .L5K format. The file may be added to a PLC program by using the Menu bar Tools - Import - Components.

FOR GRADE

Use File Explorer to show the FANUC_Robot**.L5K** file is created.

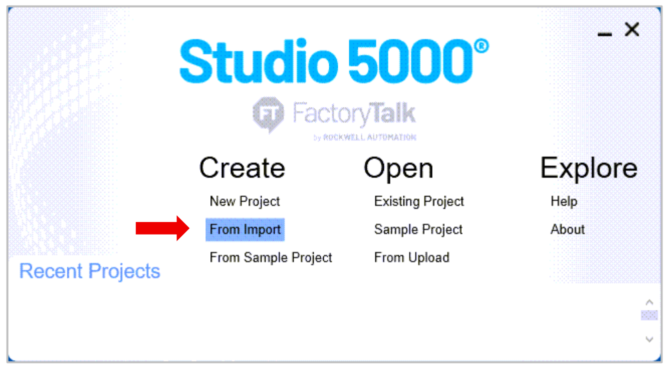
Save AS saves the whole thing

For grade show that you have created the file.

LAB: IMPORT A PLC FILE
PROGRAM FILE: FANUC_Robot.L5K
OBJECTIVE: Import the PLC program from a Library Template File.

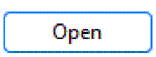
Start **Studio 5000**. 
Studio 5000

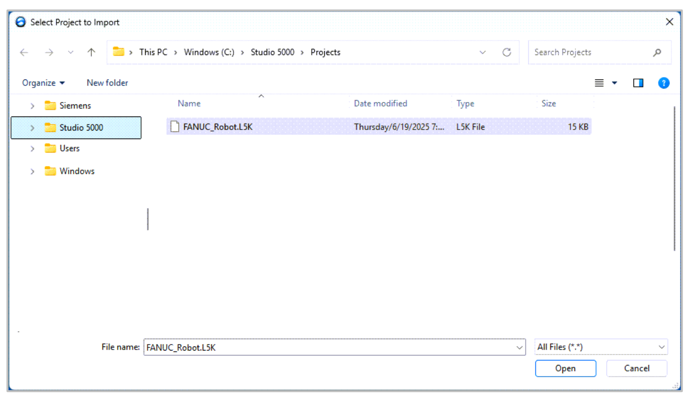
Select **From Import**.



Select the **FANUC_Robot.L5K** file.

Select the **Open** Button.

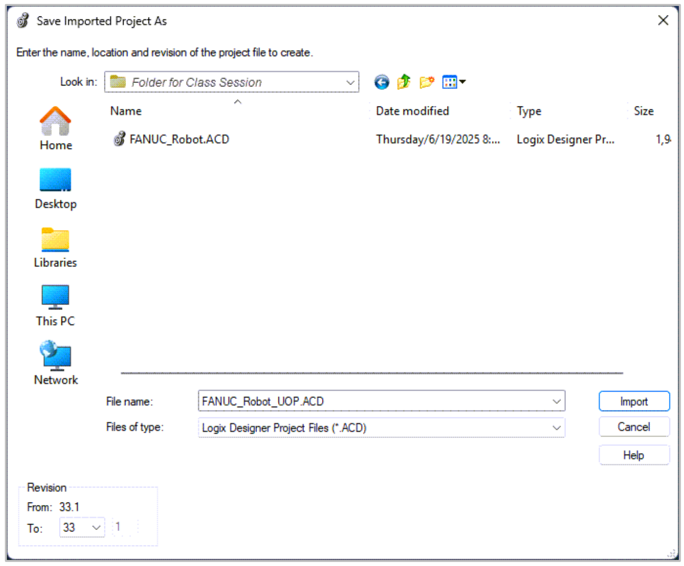




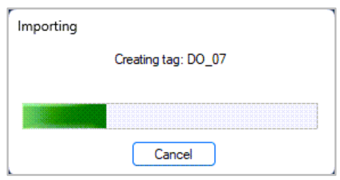
The Studio 5000 software starts.

Type the new filename
FANUC_Robot_UOP

Select the **Import** button.

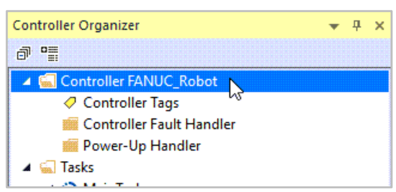


The **Importing** progress bar is shown.

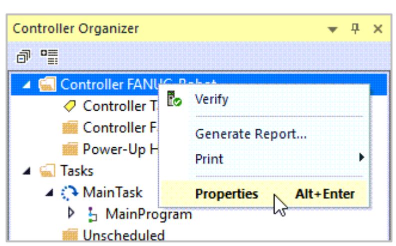


The PLC program opens.

The Controller has the original name and not the UOP project name.



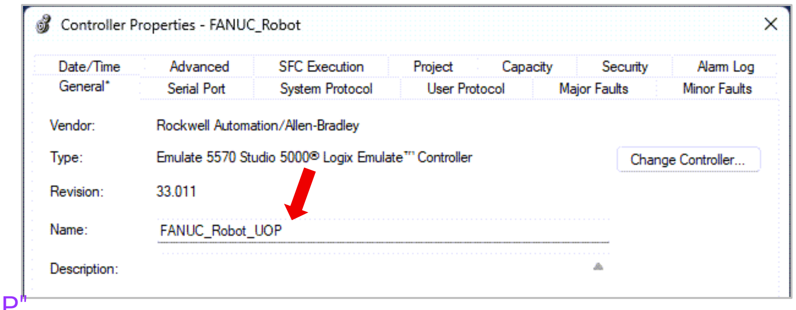
Right click on the Controller and select **Properties**.



In the Controller Properties change the controller's name.

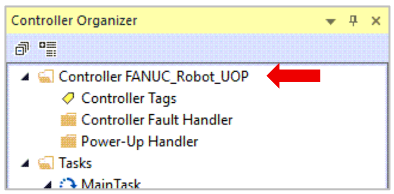
Name: FANUC_Robot_UOP

Select the **OK** button.



Add "_UOP"

The project is renamed FANUC_Robot_UOP.



Save your changes

FOR GRADE

Show the **FANUC_Robot_UOP** program.

INFORMATION ONLY

The Tag names have "Robot_1" as part of the Tag Names. If the Library .LK5 is used for a different application the Find and Replace function could be used to rename the tags, such as "Robot_2" in the new application.

Filename and name of the controller the same

Why not auto rename?

Now you have an import option under tools -> Import -> Components (could be motion groups data types etc)

Search and Replace with wildcards - that can be very useful

LAB: PROGRAMMING START AND STOP FOR ROBOT CONTROL.
PROGRAM FILE: FANUC_Robot_UOP.ACD
OBJECTIVE: Programming for the UOP START, IMSTP, and HOLD for a Robot and Machine application..

Create a basic program to control robot operations with User Operator Panel (UOP) signals for START, IMSTP, and HOLD.

UOP Signal	Description
Input: **START**	When set to ON will cause program execution to begin or resumes a paused program.
Input: **IMSTP**	The Immediate Stop is a software signal when set to OFF will stop the robot: • Pauses the program execution. • Turns Off power to the servo motors. • Applies the robot's brakes.
Input: **HOLD**	When set to OFF: • Pauses the program execution. • De-accelerates the robot's motion to a stop.
Output: **PAUSED**	This output turns ON when a program is paused. If the Teach Pendant or UOP Hold is issued the robot is paused.

The **IMSTP** and **HOLD** are normally a **Logic 1** (ON) signals when the robot is allowed to operate.

A **Logic 0** (OFF) will cause the **IMSTP** to immediately stop the robot and the **HOLD** to pause the robot as described in the table for the signals

---

The robot is tending a load/unload operation for Machine 1. The PLC based on inputs from machine coordinates the operations with the robot.

The **Machine_1** routine will have the rungs with the Input Tags that will be used to control Boolean (Internal) Output Tags for the interaction with the UOP signals.

The Boolean (Internal) Output Tags are added as Inputs to the UOP routine rungs for the corresponding control function.

---

continue

Import the Tag file: FANUC_Robot_UOP_Machine_App_Tags.CSV

Not all Imported Tags will be used in this lab. Unused Tags are for later labs. Only the Tags used are listed.

INPUT:

Alias Address	Controller Tag	Description
Machine_1:I.Data[0].0	Machine_1_MO01_Part_Loading	Loading In Progress
Machine_1:I.Data[0].1	Machine_1_MO02_Part_Unloading	Unloading In Progress
Machine_1:I.Data[0].2	Machine_1_MO03_In_Cycle	In Cycle Production
Machine_1:I.Data[0].3	Machine_1_MO04_Held	Machine Held*
Machine_1:I.Data[0].4	Machine_1_MO05_ Fault	Machine Faulted*

* Logic 1 or ON required for machine operations.

The Loading In Progress or Unloading In Progress or In Cycle Part Production are ON when the machine is actively performing the operation.

The Machine Held * and Machine Faulted * signal are ON when the machine is not Held or Faulted. When either condition is active the corresponding Output turns OFF.

OUTPUT:

Alias Address	Controller Tag	Description
Machine_1:O.Data[0].0	Machine_1_MI01_Part_Load	Load Part Request
Machine_1:O.Data[0].1	Machine_1_MI02_Part_Unloa d	Unload Part Request
Machine_1:O.Data[0].2	Machine_1_MI03_Part_Prod	Produce Part Request
Machine_1:O.Data[0].3	Machine_1_MI04_Start_Cycle	Start Request
Machine_1:O.Data[0].4	Machine_1_MI05_Hold	Machine 1 Hold Command*

* Logic 1 or ON required for machine operations.

All of the Requests are pulsed ON signals.

The machine will be held when the Hold signal turns OFF.

The Fault is Reset when the signal turns from ONOFF.

continue

INTERNAL:

Controller Tag	Data Type	Description
Robot_1_Issue_Start_Cycle	Boolean	Start Robot Command
Robot_1_Issue_Hold*	Boolean	Hold Robot Command
Machine_1_Issue_Load_Request	Boolean	Load Part Machine Command
Machine_1_Issue_Unload_Request	Boolean	Unload Part Machine Command
Machine_1_Issue_Produce_Part	Boolean	Produce Part Machine Command
Machine_1_Issue_Start_Cycle	Boolean	Start Machine Command
Machine_1_Issue_Hold*	Boolean	Hold Machine Command

* Logic 1 or ON required for machine and robot operations.

SUBROUTINES

Create the following subroutine:

Machine_1

MAIN PROGRAM

In the Main Program add an unconditional call for the subroutine **Machine_1** without any parameters.

The Main Program will already have the call to the **UOP_Robot_1** routine.

First three rungs figure out what we are doing? Load, unload, Produce

Machine_1 Routine

Interlock loading and unloading

Rung 0: **Machine_1_MI1_Part_Load**

And = series circuit

Validate check these conditions

Input Conditions:
not Machine_1_MO2_Part_Unloading **and not** Machine_1_MO3_In_Cycle **and** Machine_1_MO4_Held **and** Machine_1_MO5_Fault **and** Machine_1_Issue _Load_Request

Rung 1: **Machine_1_MI2_Part_Unload**

HMI button to initiate load request

Input Conditions:
not Machine_1_MO1_Part_Loading **and not** Machine_1_MO3_In_Cycle **and** Machine_1_MO4_Held **and** Machine_1_MO5_Fault **and** Machine_1_Issue _Unload_Request

This takes less time to get back moving

First three rungs check valid ?

Not Loading , not unloading not in cycle and fault is low
internal is low = go ahead and produce part

If in cycle we can't start a cycle, it's already running

Rung 2: **Machine_1_MI3_Part_Prod**

Input Conditions:

not Machine_1_MO1_Part_Loading **and not** Machine_1_MO2_Part_Unloading **and not** Machine_1_MO3_In_Cycle **and** Machine_1_MO4_Held **and** Machine_1_MO5_Fault **and** Machine_1_Issue_Produce_Part

Rung 3: **Machine_1_MI4_Start_Cycle** Interlocked to rung2

Lot's of interlocks check conditions

Input Conditions:

not Machine_1_MO3_In_Cycle and Machine_1_MO4_Held **and** Machine_1_MO5_Fault **and not** Robot_1_UO04_PAUSED **and** Machine_1_MI1_Part_Load **or** Machine_1_MI2 _Part_Unload **or** Machine_1_MI3_Part_Prod **and** Machine_1_Issue_Start_Cycle

Or = parallel circuits

HOLD High input signals if go low

Rung 4: **Machine_1_MI5_Hold**

Input Conditions:

Machine_1_Issue_Hold **and** Robot_1_Issue_Hold

INFORMATION ON MACHINE START

The checks for the Machine to Load, Unload, or Produce the Part are in this Machine routine.

Control stop input

Rungs 0 to 2:

The Output signals to Load, Unload, or Produce the Part are dependent on the machine is not In Cycle (this prevents a conflicting command if the machine is already actively loading, unloading, or producing a part). The machine is not Held or Faulted.

The Load and Unload are interlocked to prevent both commands from being issued.

If the above conditions are satisfied, a machine Issue Request is set (ON) for the Load, Unload, or Produce Parts outputs.

Finish current program before it stops

Rung 3:

If the machine is commanded to Start if it is not currently In Cycle, and there are no active Machine Hold or Fault, and the Robot is not Paused, and there is a valid Unload, Load or Produce Part (ON), a machine Issue Cycle Start is set (ON).

Rung 4:

If a Machine or Robot Hold is issued, the rung must be False to cause the machine to halt operations.

Example sealant gun sits with sealant in nozzle.
Need to tell robot to finish emptying nozzle first

continue

When turns off resets the fault. Initially on.

UOP_Robot_1 Routine

Drag in a **New Rung 5** for the **UOP START** signal.

Rung 0: **Robot_1_UI01_IMSTP**
Input Conditions:
Machine_1_MO5_Fault

Rung 1: **Robot_1_UI02_HOLD**
Input Conditions:
Robot_1_Issue_Hold **and** Machine_1_MI5_Hold

Rung 2: **Robot_1_UI03_SFSPD**
Input Conditions:
None

Enable,
if off can't move
enable motion

Rung 3: **Robot_1_UI04_CSTOPI**
Input Conditions:
None

Rung 4: **Robot_1_UI05_FAULT_RESET**
Input Conditions:
None

Rung 5: **Robot_1_UI06_START**
Input Conditions:
Machine_1_MO4_Held **and** Machine_1 _MO5_Fault **and** Machine_1_MO03 _In_Cycle **and** Robot_1_Issue_Start_Cycle

Rung 6: **Robot_1_UI08_ENBL**
Input Conditions:
None

INFORMATION ON STOP, HOLD, and START

Rung 0: If the machine issues a Fault the robot will Stop and Pause program execution. A fault is a serious problem, and the robot stops immediately and power to the motors is turned Off.

Rung 1: If the machine is Held the robot is Held and Pauses the program. In HOLD the power to the robot's motor will be present, usually for a predetermined time, to allow the robot to continue motion if the hold is for a short time.

Rung 5: A valid robot Start is when there are no Machine Fault or Hold

The checks for a valid Load Part, or Unload Part, or Produce Part are in the Machine routine.

The Machine 1 In Cycle confirms the machine has accepted an operation (Load, Unload, or Produce) command.

A robot Issue Cycle Start is set (ON) to execute the robot operation.

The check for the Machine is then Held or Faulted are included as conditions for the Robot Start because the machine is not operational when those conditions exist. See Rungs 0 and 1 for the signal to cause the robot's motion to be suspended.

FOR GRADE

Show the **UOP_Robot_1** routine and **Machine_1** routine.

No Downloading yet

LAB: PROGRAMMING FAULT MONITORING AND RESET
PROGRAM FILE: FANUC_Robot_UOP
OBJECTIVE: Programming for the UOP FAULT and RESET .

Create a basic program to control robot operations with User Operator Panel (UOP) signals for FAULT and FAULT RESET. Usually fault reset is human intervention

UOP Signal	Description
Input: **FAULT_RESET**	This input is the external fault reset signal. When this signal transitions from ON to OFF the following will happen: • Error status is cleared • Servo power is turned on • The paused program ***will not be resumed***
Output: **FAULT**	This output is the error output. This output turns ON when the controller/ program is in an error condition

The **FAULT_RESET** is normally a **Logic 1** (ON) signals when the robot is allowed to operate.

The **FAULT_RESET** must transition from ON to OFF to allow the fault to be cleared.

Already imported from previous labs

INPUT:

Alias Address	Controller Tag	Description
Machine_1:I.Data[0].4	Machine_1_MO05_Fault	Machine 1 Faulted

OUTPUT:

Alias Address	Controller Tag	Description
Machine_1:O.Data[0].5	Machine_1_MI06_Fault_Reset	Machine 1 Reset Fault*

* Logic 1 or ON required for machine operations.

INTERNALS:

Controller Tag	Data Type	Description
Robot_1_ Issue_Fault_Reset	Boolean	Fault Reset Command
Machine_1_Issue_Fault_Reset	Boolean	Fault Reset Command

UOP_Robot_1 Routine

Rung 4: **Robot_1_UI05_FAULT_RESET**
    Input Conditions:
    Robot_1_Issue_Fault_Reset

INFORMATION ON FAULT RESET

Rung 4 must go from True (Robot_1_Issue_Fault_Reset ON) to False (Robot_1_Issue_Fault_Reset OFF) for the robot controller to clear the error condition.

Machine_1 Routine

Rung 3: **Machine_1_MI04_Start_Cycle**
    Input Conditions:
    Faulted
    Add in series (*before parallel branches*) Robot_1_UO04_~~PAUSED~~

The Machine Fault Reset is similar to the Robot; the rung must go from True to False for the error to be cleared.

New Rung 5: **Machine_1_MI06_Fault_Reset**
    Input Conditions:
    Machine_1_Issue_Fault_Reset

INFORMATION ON ROBOT FAULT

Rung 3 if the Robot is Faulted a part Load or Unload cannot be performed.

FOR GRADE

Show the changes to the **UOP_Robot_1** routine and **Machine_1** routine.

LAB: PROGRAMMING STATUS FOR READY TO RUN\RUNNING
PROGRAM FILE: FANUC_Robot_UOP
OBJECTIVE: Programming for the UOP signals that are needed for a Ready to Run status and Running status .

Create a basic program to control robot operations with User Operator Panel (UOP) signals to determine if the robot is Ready to Run or is Running.

UOP Signal	Description
Output: **CMDENBL**	This is the command enable output. This output indicates that the robot is in a remote condition. This output only stays on when the robot is not in a fault condition. When SYSRDY is OFF, CMDENBL is OFF. This signal goes on when the following conditions are **all satisfied**: • Teach pendant disabled • Mode selection switch is set to AUTO • Robot set to Remote • *IMSTP input is ON • *HOLD input is ON • *SFSPD input is ON • *CSTOPI input is ON • **ENBL input is ON
Output: **SYSRDY**	This output is the system ready output. This output indicates that the system can provide power for the servo motors.
Output: **PROGRUN**	This output turns on when a program is running.

INTERNALS:

Controller Tag	Data Type	Description
Robot_1_Status_Ready_Auto	Boolean	Fault Reset Command
Robot_1_Status_System_Ready	Boolean	Fault Reset Requested
Robot_1_Status_Running_Program	Boolean	One Shot Array for Robot

UOP_Robot_1 Routine

New Rung 7: Robot_1_Status_Ready_Auto

Input Conditions:

Robot_1_UO01_CMDENBL

New Rung 8: Robot_1_Status_System_Ready

Input Conditions:

Robot_1_UO02_SYSRDY

New Rung 9: Robot_1_Status_Running_Program

Input Conditions:

Robot_1_UO03_PROGRUN

INFORMATION ON ROBOT STATUS SIGNALS

The **UOP Signal**, *see Table*, could be used directly for the HMI or other PLC logic. The UOP names are not descriptive for someone who does not know what the signals are indicating. By assigning the UOP signals to PLC Tags a more descriptive name is used.

As an alternative to the added rungs, an **Alias** for UOP Signal could be set up in the Tag Editor.

Name	Alias For	Base Tag
Robot_1_Status_Ready_Auto	Robot_1_UO01_CMDENBL	Robot_1:I1.Input[0].0
Robot_1_Status_Running_Program	Robot_1_UO03_PROGRUN	Robot_1:I1.Input[0].2
Robot_1_Status_System_Ready	Robot_1_UO02_SYSRDY	Robot_1:I1.Input[0].1

APPENDIX A: I/O ASSIGNMENTS

ROBOT INPUTS	ASSIGNMENTS	PLC OUTPUTS	TAGS
UI[1]	*IMSTP	Robot_1:O1.Output[0].0	Robot_1_UI01_IMSTP
UI[2]	*HOLD	Robot_1:O1.Output[0].1	Robot_1_UI02_HOLD
UI[3]	*SFSPD	Robot_1:O1.Output[0].2	Robot_1_UI03_SFSPD
UI[4]	*CSTOPI	Robot_1:O1.Output[0].3	Robot_1_UI04_CSTOPI
UI[5]	*FAULT RESET	Robot_1:O1.Output[0].4	Robot_1_UI05_FAULT_RESET
UI[6]	START	Robot_1:O1.Output[0].5	Robot_1_UI06_START
UI[7]	HOME	Robot_1:O1.Output[0].6	Robot_1_UI07_HOME
UI[8]	**ENBL	Robot_1:O1.Output[0].7	Robot_1_UI08_ENBL
UI[9]	STYLE1/RSR1/PNS1	Robot_1:O1.Output[1].0	Robot_1_UI09_STYLE1_RSR1_PNS1
UI[10]	STYLE2/RSR2/PNS2	Robot_1:O1.Output[1].1	Robot_1_UI10_STYLE2_RSR2_PNS2
UI[11]	STYLE3/RSR3/PNS3	Robot_1:O1.Output[1].2	Robot_1_UI11_STYLE3_RSR3_PNS3
UI[12]	STYLE4/RSR4/PNS4	Robot_1:O1.Output[1].3	Robot_1_UI12_STYLE4_RSR4_PNS4
UI[13]	STYLE5/PNS5/RSR5	Robot_1:O1.Output[1].4	Robot_1_UI13_STYLE5_PNS5_RSR5
UI[14]	STYLE6/PNS6/RSR6	Robot_1:O1.Output[1].5	Robot_1_UI14_STYLE6_PNS6_RSR6
UI[15]	STYLE7/PNS7/RSR7	Robot_1:O1.Output[1].6	Robot_1_UI15_STYLE7_PNS7_RSR7
UI[16]	STYLE8/PNS8/RSR8	Robot_1:O1.Output[1].7	Robot_1_UI16_STYLE8_PNS8_RSR8
UI[17]	PNSTROBE	Robot_1:O1.Output[2].0	Robot_1_UI17_PNSTROBE
UI[18]	PROD_START	Robot_1:O1.Output[2].1	Robot_1_UI18_PROD_START
DI[1]	DIGITAL INPUT 1	Robot_1:O1.Output[2].2	Robot_1_DI_01
DI[2]	DIGITAL INPUT 2	Robot_1:O1.Output[2].3	Robot_1_DI_02
DI[3]	DIGITAL INPUT 3	Robot_1:O1.Output[2].4	Robot_1_DI_03
DI[4]	DIGITAL INPUT 4	Robot_1:O1.Output[2].5	Robot_1_DI_04
DI[5]	DIGITAL INPUT 5	Robot_1:O1.Output[2].6	Robot_1_DI_05
DI[6]	DIGITAL INPUT 6	Robot_1:O1.Output[2].7	Robot_1_DI_06
DI[7]	DIGITAL INPUT 7	Robot_1:O1.Output[3].0	Robot_1_DI_07
DI[8]	DIGITAL INPUT 8	Robot_1:O1.Output[3].1	Robot_1_DI_08
DI[9]	GROUP INPUT 1	Robot_1:O1.Output[3].2	Robot_1_GI_01_Binary1
DI[10]	GROUP INPUT 1	Robot_1:O1.Output[3].3	Robot_1_GI_01_Binary2
DI[11]	GROUP INPUT 1	Robot_1:O1.Output[3].4	Robot_1_GI_01_Binary4
DI[12]	GROUP INPUT 1	Robot_1:O1.Output[3].5	Robot_1_GI_01_Binary8
DI[13]	UNUSED	Robot_1:O1.Output[3].6	UNUSED_IN1
DI[14]	UNUSED	Robot_1:O1.Output[3].7	UNUSED_IN2

ROBOT OUTPUTS	ASIGNMENTS	PLC INPUTS	TAGS
UO[1]	CMDENBL	Robot_1:I1.Input[0].0	Robot_1_UO01_CMDENBL
UO[2]	SYSRDY	Robot_1:I1.Input[0].1	Robot_1_UO02_SYSRDY
UO[3]	PROGRUN	Robot_1:I1.Input[0].2	Robot_1_UO03_PROGRUN
UO[4]	PAUSED	Robot_1:I1.Input[0].3	Robot_1_UO04_PAUSED
UO[5]	HELD	Robot_1:I1.Input[0].4	Robot_1_UO05_HELD
UO[6]	FAULT	Robot_1:I1.Input[0].5	Robot_1_UO06_FAULT
UO[7]	ATPERCH	Robot_1:I1.Input[0].6	Robot_1_UO07_ATPERCH
UO[8]	TPENBL	Robot_1:I1.Input[0].7	Robot_1_UO08_TPENBL
UO[9]	BATALM	Robot_1:I1.Input[1].0	Robot_1_UO09_BATALM
UO[10]	BUSY	Robot_1:I1.Input[1].1	Robot_1_UO10_BUSY
UO[11]	STYLE1/ACK1/SNO1	Robot_1:I1.Input[1].2	Robot_1_UO11_STYLE1_ACK1_SNO1
UO[12]	STYLE2/ACK2/SNO2	Robot_1:I1.Input[1].3	Robot_1_UO12_STYLE2_ACK2_SNO2
UO[13]	STYLE3/ACK3/SNO3	Robot_1:I1.Input[1].4	Robot_1_UO13_STYLE3_ACK3_SNO3
UO[14]	STYLE4/ACK4/SNO4	Robot_1:I1.Input[1].5	Robot_1_UO14_STYLE4_ACK4_SNO4
UO[15]	STYLE5/SNO5/ACK5	Robot_1:I1.Input[1].6	Robot_1_UO15_STYLE5_SNO5_ACK5
UO[16]	STYLE6/SNO6/ACK6	Robot_1:I1.Input[1].7	Robot_1_UO16_STYLE6_SNO6_ACK6
UO[17]	STYLE7/SNO7/ACK7	Robot_1:I1.Input[2].0	Robot_1_UO17_STYLE7_SNO7_ACK7
UO[18]	STYLE8/SNO8/ACK8	Robot_1:I1.Input[2].1	Robot_1_UO18_STYLE8_SNO8_ACK8
UO[19]	SNACK	Robot_1:I1.Input[2].2	Robot_1_UO19_SNACK
UO[20]	RESERVED	Robot_1:I1.Input[2].3	Robot_1_UO20_RESERVED
DO[1]	DIGITAL OUTPUT 1	Robot_1:I1.Input[2].4	Robot_1_DO_01
DO[2]	DIGITAL OUTPUT 2	Robot_1:I1.Input[2].5	Robot_1_DO_02
DO[3]	DIGITAL OUTPUT 3	Robot_1:I1.Input[2].6	Robot_1_DO_03
DO[4]	DIGITAL OUTPUT 4	Robot_1:I1.Input[2].7	Robot_1_DO_04
DO[5]	DIGITAL OUTPUT 5	Robot_1:I1.Input[3].0	Robot_1_DO_05
DO[6]	DIGITAL OUTPUT 6	Robot_1:I1.Input[3].1	Robot_1_DO_06
DO[7]	DIGITAL OUTPUT 7	Robot_1:I1.Input[3].2	Robot_1_DO_07
DO[8]	DIGITAL OUTPUT 8	Robot_1:I1.Input[3].3	Robot_1_DO_08
DO[9]	GROUP OUTPUT 1	Robot_1:I1.Input[3].4	Robot_1_GO_01_Binary1
DO[10]	GROUP OUTPUT 1	Robot_1:I1.Input[3].5	Robot_1_GO_01_Binary2
DO[11]	GROUP OUTPUT 1	Robot_1:I1.Input[3].6	Robot_1_GO_01_Binary4
DO[12]	GROUP OUTPUT 1	Robot_1:I1.Input[3].7	Robot_1_GO_01_Binary8

APPENDIX B: FANUC USER OPERATOR PANEL ASSIGNMENTS

USER OPERATOR PANEL (UOP) INPUTS

UOP Input Signals	UOP UI
*IMSTP	UI 1
*HOLD	UI 2
*SFSPD	UI 3
*CSTOPI (1)	UI 4
*FAULT RESET(1)	UI 5
START	UI 6
HOME	UI 7
**ENBL	UI 8
STYLE/RSR1/PNS1	UI 9
STYLE/RSR2/PNS2	UI 10
STYLE/RSR3/PNS3	UI 11
STYLE/RSR4/PNS4	UI 12
STYLE/PNS5/RSR5(2)	UI 13
STYLE/PNS6/RSR6(2)	UI 14
STYLE/PNS7/RSR7(2)	UI 15
STYLE/PNS8/RSR8(2)	UI 16
PNSTROBE	UI 17
PROD_START	UI 18

* Logic 1 or ON signal required for robot operations.

(1) Signals require a transition to be active.

** Logic 1 or ON signal required for robot operations.

(2) If a second Motion Group is used.

UOP Input Signal	Description
***IMSTP** Held ON signal UI[1]	This input is the immediate stop software signal. *IMSTP is a signal held ON. When it is set to OFF, it • Pauses the program. • Immediately stops the robot and applies robot brakes • Shuts off power to the servos **WARNING** *IMSTP is a software controlled input and cannot be used for safety purposes.
***HOLD** Held ON signal UI[2]	This input is the external hold signal. *Hold is a signal held ON. When it is set to OFF, it will do the following: • Pause program execution • Slow motion to a controlled stop and hold • Does not shut off servo power.

UOP Input Signal	Description
***SFSPD** Held ON signal UI[3]	This input is the safety speed input signal. This signal is usually connected to the safety fence. *SFSPD is a signal held ON. When it is set to OFF it will do the following: • Pause program execution • Reduce the speed override value to that defined in a system variable. This value cannot be increased while *SFSPD is OFF. • Display error code message SYST009. • Not allow is a REMOTE start condition for robot motion. Start inputs from UOP or SOP are disabled when SFSPD is set to OFF and only the teach pendant has motion control with the speed clamped.
***CSTOPI** Held ON signal UI[4]	This input is the cycle stop input. *CSTOPI is a signal held ON. When it is set to OFF, it will do the following: The function of this signal depends on the system variable $SHELL_CFG.$USE_ABORT. **$SHELL_CFG.$USE_ABORT** = **FALSE** **WARNING** **$SHELL_CFG.$USE_ABORT = FALSE**, CSTOPI does not immediately stop automatic program execution. Automatic execution **will be stopped after the current program has finished executing.** • Clears the queue of programs to be executed that were sent by RSR signals **$SHELL_CFG.$USE_ABORT** = **TRUE** • Immediately aborts the currently executing program(s) for programs that were sent to be executed by either RSR or PNS. • Clears the queue of programs to be executed that were sent by RSR signals.

UOP Input Signal	Description
*** FAULT_RESET** Held ON signal UI[5]	This input is the external fault reset signal. When this signal transitions from ON to OFF the following will happen: • Error status is cleared • Servo power is turned on • The paused program ***will not be resumed***
START (CMDENBL = ON) UI[6]	This input is the remote start input. The function of this signal depends on the system variable $SHELL_CFG.$CONT_ONLY. **$SHELL_CFG.$CONT_ONLY** = **FALSE** • Resumes a Paused program • If a program is Aborted, the currently selected program starts from the position of the cursor. **$SHELL_CFG.$CONT_ONLY** = **TRUE** • START resumes a Paused program only. • *The* PROD_START *input must be used to start an Aborted program from the beginning.*
HOME (CMDENBL = ON) UI[7]	This input is the home input. When this signal is received the robot moves to the defined home position. You configure the system to do this by setting up a macro program to run when UI[7] is received.
****ENBL** Held ON signal UI[8]	This input is the enable input. This signal must be ON to have motion control ability. When this signal is OFF, robot motion cannot be done. When ENBL is ON and the REMOTE switch on the operator panel is in the REMOTE position, the robot is in a remote operating condition.

UOP Input Signal	Description
STYLE 1-8 **RSR 1-8** **PNS 1-8** (CMDENBL = ON) UI[9] - UI[16]	Signals are Binary Numbers read as an Integer value. The inputs select the Program Number. Style or PNS selects programs for execution but ***does not execute programs*.** The selected Style or PNS Programs are executed using the START input or the PROD_START input. See $SHELL_CFG.$CONT_ONLY. RSR programs are queued and execute in order queued. Type of signal depends on Production Select Method.
PNSTROBE (CMDENBL = ON) UI[17]	PNS only, a strobe signal to read Program Number Select UI[9] – UI[16] input signals.
PROD_START Active when the robot is in a remote condition (CMDENBL = ON) UI[18]	This item is the Production Start Input when used with STYLE or PNS to initiate execution of the selected Program Number UI[9] – UI[16].

USER OPERATOR PANEL (UOP) OUTPUTS

UOP Output Signals	UOP UO
CMDENBL	UO 1
SYSRDY	UO 2
PROGRUN	UO 3
PAUSED	UO 4
HELD	UO 5
FAULT	UO 6
ATPERCH	UO 7
TPENBL	UO 8
BATALM	UO 9
BUSY	UO 10
STYLE/ACK1/SNO1	UO 11
STYLE/ACK2/SNO2	UO 12
STYLE/ACK3/SNO3	UO 13
STYLE/ACK4/SNO4	UO 14
STYLE/SNO5/ACK5(1)	UO 15
STYLE/SNO6/ACK6(1)	UO 16
STYLE/SNO7/ACK7(1)	UO 17
STYLE/SNO8/ACK8(1)	UO 18
SNACK	UO 19
RESERVED	UO 20

(1) If a second Motion Group is used.

UOP Output Signal	Description
CMDENBL UO[1]	This is the command enable output. This output indicates that the robot is in a remote condition. This output only stays on when the robot is not in a fault condition. When SYSRDY is OFF, CMDENBL is OFF. This signal goes on when the following conditions are **all satisfied**: • Teach pendant disabled • Mode selection switch is set to AUTO • Robot set to Remote • *IMSTP input is ON • *HOLD input is ON • *SFSPD input is ON • *CSTOPI input is ON • **ENBL input is ON

UOP Output Signal	Description
SYSRDY UO[2]	This output is the system ready output. This output indicates that the system can provide power for the servo motors.
PROGRUN UO[3]	This output turns on when a program is running.
PAUSED UO[4]	This output is the paused program output. This output turns ON when a program is paused. If the Teach Pendant or UOP Hold is issued the robot is paused.
HELD UO[5]	This output is the hold output. This output turns on when the SOP HOLD button has been pressed, or the UOP *HOLD input is OFF.
FAULT UO[6]	This output is the error output. This output turns on when the controller/program is in an error condition.
ATPERCH UO[7]	This output is the at perch output. This output turns on when the robot reaches the predefined perch position. **ATPERCH position = Reference position assigned UO[7]**.
TPENBL UO[8]	This output is the teach pendant enable output. This output turns on when the teach pendant is on and keyswitch is in Teach mode.
BATALM UO[9]	This output is the battery alarm output. This output turns on when the CMOS RAM battery voltage goes below 2.6 volts.
BUSY	This output is the processor busy output. This signal turns on when the robot is executing a program or when the processor is busy.

UO[10]	

UOP Output Signal	Description
STYLE 1-8 **ACK 1-8** **SNO 1-8** UO[11] - UO[18]	Signals are Binary Numbers read as an Integer value. These outputs signal the selected Program Number.
SNACK UO[19]	PNS only. This output is the PNS program number acknowledge output.
RESERVED UO[20]	Unassigned